Practical 1: Sorting Implementation and Analysis

Student Name: Tushar Pirwani
Student ID: 22464293
Unit: Data Structures and Algorithms (COMP1002)

Activity 4: Exploring Sorting Runtimes

This section presents the results of the performance investigation into the three implemented sorting algorithms.

Runtime Results Table

The following table shows the average runtime in seconds for each sorting algorithm across four different array sizes and three initial data orderings.

Algorithm	Array Type	Array Size	Runtime (seconds)
Bubble Sort	Random (r)	100	0.00556
Bubble Sort	Random (r)	500	0.05879
Bubble Sort	Random (r)	1000	0.20102
Bubble Sort	Random (r)	2000	0.70838
Selection Sort	Random (r)	100	0.01018
Selection Sort	Random (r)	500	0.05190
Selection Sort	Random (r)	1000	0.11969
Selection Sort	Random (r)	2000	0.37787
Insertion Sort	Random (r)	100	0.00834
Insertion Sort	Random (r)	500	0.05259
Insertion Sort	Random (r)	1000	0.16429
Insertion Sort	Random (r)	2000	0.56995
---	---	---	---
Bubble Sort	Descending (d)	1000	0.22005
Selection Sort	Descending (d)	1000	0.07575
Insertion Sort	Descending (d)	1000	0.23140
---	---	---	---
Bubble Sort	Nearly Sorted (n)	1000	0.08616
Selection Sort	Nearly Sorted (n)	1000	0.07507
Insertion Sort	Nearly Sorted (n)	1000	0.01665

Discussion of Results

The experimental results from testing Bubble, Selection, and Insertion sorting Algorithms reveal critical insights into their performance characteristics and practical use cases. A key observation is the poor **scalability** of all three algorithms. When analyzing their performance on random data, it's clear that the runtime does not increase linearly with the input size. For instance, with Bubble Sort, when the array size doubled from 1000 to 2000, the runtime increased from approximately 0.20 seconds to 0.71 seconds, a jump of about 3.5 times. This quadratic growth is consistent with the $O(n^2)$ average-case **time complexity** expected from these algorithms.

The most significant differences emerge when comparing the **best, average, and worst-case** scenarios using the data for an array size of 1000. **Selection Sort** demonstrated remarkable consistency, with its runtime for descending data (0.075s) being very close to its time for nearly sorted data (0.075s). This is because it always performs the same number of comparisons to find the minimum value, regardless of the initial order. In contrast, **Bubble Sort** and **Insertion Sort** are highly adaptive. Their worst-case performance occurred with descending data (0.22s and 0.23s, respectively), where every element is in the wrong position. However, their best-case performance on nearly sorted data was exceptionally fast. Insertion Sort, in particular, was the clear winner in this scenario, taking only 0.016s, which is significantly faster than Bubble Sort's 0.086s. This highlights the effectiveness of the "early exit" optimization in Bubble Sort and the minimal shuffling required by Insertion Sort when data is mostly ordered. In conclusion, while none of these algorithms are suitable for large, random datasets, the data shows that Insertion Sort is by far the most efficient choice for lists that are already nearly sorted.